

Replay4NCL: An Efficient Memory Replay-based Methodology for Neuromorphic Continual Learning in Embedded AI Systems

Mishal Fatima Minhas¹, Rachmad Vidya Wicaksana Putra², Falah Awwad¹, Osman Hasan³, Muhammad Shafique²

¹Electrical and Communication Engineering Department, United Arab Emirates University (UAEU), Al Ain, UAE

²eBrain Lab, New York University (NYU) Abu Dhabi, Abu Dhabi, UAE

³School of Electrical Engineering and Computer Science (SEecs), National University of Sciences and Technology (NUST), Islamabad, Pakistan

{mishal.fatima, f_awwad}@uaeu.ac.ae, {rachmad.putra, muhammad.shafique}@nyu.edu, osman.hasan@seecs.edu.pk

Abstract—Neuromorphic Continual Learning (NCL) paradigm leverages Spiking Neural Networks (SNNs) to enable continual learning (CL) capabilities for AI systems to adapt to dynamically changing environments. Currently, the state-of-the-art employ a memory replay-based method to maintain the old knowledge. However, this technique relies on long timesteps and compression-decompression steps, thereby incurring significant latency and energy overheads, which are not suitable for tightly-constrained embedded AI systems (e.g., mobile agents/robotics). To address this, we propose *Replay4NCL*, a novel efficient memory replay-based methodology for enabling NCL in embedded AI systems. Specifically, *Replay4NCL* compresses the latent data (old knowledge), then replays them during the NCL training phase with small timesteps, to minimize the processing latency and energy consumption. To compensate the information loss from reduced spikes, we adjust the neuron threshold potential and learning rate settings. Experimental results on the class-incremental scenario with the Spiking Heidelberg Digits (SHD) dataset show that *Replay4NCL* can preserve old knowledge with Top-1 accuracy of 90.43% compared to 86.22% from the state-of-the-art, while effectively learning new tasks, achieving 4.88x latency speed-up, 20% latent memory saving, and 36.43% energy saving. These results highlight the potential of our *Replay4NCL* methodology to further advances NCL capabilities for embedded AI systems.

Index Terms—Spiking Neural Networks, Neuromorphic Continual Learning, Memory Replay, Efficiency, Embedded AI

I. INTRODUCTION

The continual learning (CL) capability shown by human has inspired the developments of CL for artificial intelligence (AI) systems, e.g., through Deep Neural Networks (DNNs). CL capability is highly desired by systems that are supposed to adapt to dynamically changing environments [1]–[3], since the knowledge acquired during the off-line training (pre-training) phase may become outdated over time. This condition can result in reduced accuracy at run-time. To address this, the knowledge update is required to ensure correct functionality of the system. However, simply performing the retraining technique from scratch is not practical, as it requires huge training time and energy consumption. Besides, it also requires a full dataset including data for the new task, which is not always available due to data restriction policy [4]. Moreover, if the retraining is not performed carefully, it may also cause the *catastrophic forgetting* (CF) issue, where accuracy for the previously trained tasks significantly decrease as the system learns a new task [5]–[10], as illustrated in Fig. 1(a).

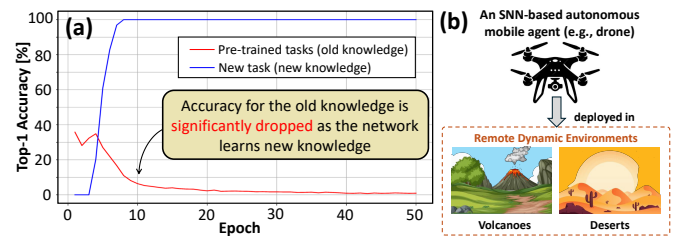


Fig. 1. (a) Accuracy of the baseline network that faces CF issues in the class-incremental learning scenario. Baseline network refers to an SNN model without any NCL capabilities. (b) Illustration of the use-case of an embedded AI system that requires energy-efficient CL capabilities, e.g., a mobile agent.

Toward this, many researches have proposed different CL methods for DNNs [2]. However, the intensive DNN operations make it challenging to efficiently enable CL capabilities, especially in the application use-cases where embedded AI systems with limited battery capacity and resources are deployed in environments that are too far or not safe for humans (e.g., mobile agents/robots like UAVs and UGVs) [11], as shown in Fig. 1(b).

In recent years, *neuromorphic continual learning* (NCL) has emerged as an attractive approach to enable efficient CL capabilities, i.e., by leveraging Spiking Neural Networks (SNNs) to perform sparse spike-based CL algorithms [12]–[17]. However, the NCL field is still very new, thereby practical solutions and methodologies for enabling efficient NCL capabilities in embedded AI systems is still not comprehensively explored. Therefore, **we mainly target the following research problem** in this paper, *how can we enable efficient NCL capabilities for resource-constrained embedded AI systems?* An efficient solution to this problem potentially enables the efficient NCL deployments for diverse applications use-cases.

A. State-of-the-Art and Their Limitations

To address CF with NCL paradigm, previous works can be categorized into two different approaches based on the learning settings, i.e., *unsupervised*- and *supervised-based learning*.

In the unsupervised learning settings, previous works have proposed different techniques, such as enhancements on bio-plausible Spike-Timing Dependent Plasticity (STDP) learning rules [13]–[16] and predictive coding [18]. However, these studies still consider static and non-event-based datasets in

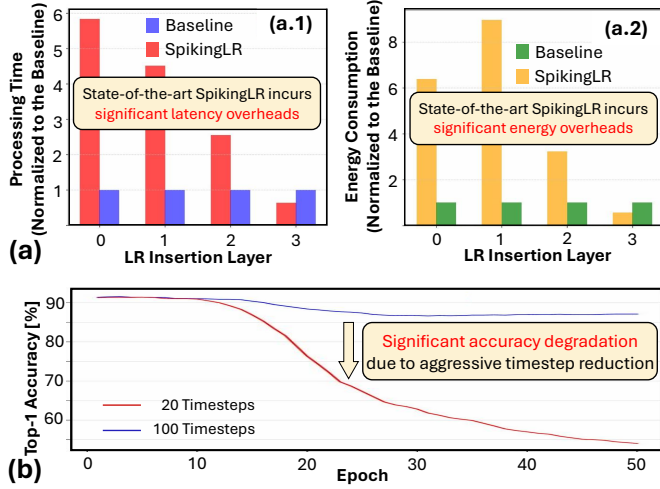


Fig. 2. (a) The state-of-the-art memory replay-based work, SpikingLR [25], incurs significant latency and energy overheads as compared to the baseline network without NCL techniques. (b) An aggressive timestep reduction (e.g., from 100 to 20 timesteps) may lead to significant accuracy degradation.

their evaluations, e.g., MNIST [19] and Fashion MNIST [20]. Therefore, their applicability for event-based neuromorphic data (e.g., from dynamic vision sensors) is still not evaluated.

Meanwhile, in the supervised learning settings, recent works have also proposed different techniques, such as active dendrites [21], architectural enhancements [22] [23], memory replay [24] [25], regularization [26], as well as Bayesian [27] and Hebbian [28] learning. Currently, the state-of-the-art work employs the memory replay-based technique to preserve high accuracy on the old knowledge (i.e., *latent knowledge*) while also learning new knowledge [25]; so-called the *spiking latent replay* (*SpikingLR*) for brevity. However, this technique relies on long timesteps and compression-decompression steps to achieve high accuracy [25], hence incurring significant latency and energy consumption overheads, which are not suitable for tightly-constrained embedded AI systems. These limitations indicate that *an alternate efficient memory replay-based technique is highly needed to provide NCL capabilities with low memory, latency, and energy overheads*.

To illustrate the limitations of state-of-the-art and the optimization challenges, we perform an experimental case study, which is discussed further in Section I-B.

B. Case Study and Associated Research Challenges

In this case study, we investigate the impact of a widely-used optimization technique for SNNs (i.e., timestep reduction [29]), while employing the memory replay-based technique in the class-incremental scenario. To do this, we consider the implementation of SpikingLR [25], and perform a timestep reduction accordingly (e.g., from 100 to 20 timesteps). Details of the experimental setup are provided in Section IV. The experimental results are presented in Fig. 2, from which we identify the following key observations.

- The state-of-the-art Spiking-LR work introduces huge latency and energy overheads due to its long timesteps.
- The timestep reduction may degrade the accuracy significantly, thereby hindering the implementation of optimizations on memory replay-based techniques for NCL.

Furthermore, this observation exposes several research challenges that need to be addressed, as follows.

- The processing latency should be optimized carefully so that the system can quickly respond properly to the given inputs, without a significant accuracy degradation.
- The enhancement techniques are required to compensate the loss of information due to the reduced timestep.

C. Our Novel Contributions

To address the above challenges, we propose **Replay4NCL**, a novel methodology to enable efficient memory Replay-based NCL in resource-constrained embedded AI systems. To achieve this, our Replay4NCL employs the following novel key design steps; see an overview in Fig. 4.

- **Perform Timestep Optimization (Section III-A):** It aims to introduce a low timestep setting into the SNN model to substantially reduce the processing latency and minimize the latent data memory.
- **Perform Parameter Adjustments (Section III-B):** It aims to compensate the loss of information due to the timestep reduction, by adjusting the parameters (e.g., neuron threshold potential and learning rate) to adapt to fewer spikes in the old (latent) data.
- **Devise a Data Insertion Strategy (Section III-C):** It aims to define the layer that will receive the latent data, so that it can effectively lead to high learning quality of latent data, and hence avoiding the CF problem.

Key Results: Our proposed methodology is evaluated using a Python implementation that runs on the Nvidia RTX 4090 Ti GPU machines, while considering the SHD dataset [30]. The experimental results on the class-incremental scenario show that our Replay4NCL maintains old knowledge with Top-1 accuracy of 90.43% compared to 86.22% from the state-of-the-art SpikingLR, while effectively learning new tasks, improving the latency by 4.88x, saving latent memory by 20%, and saving energy consumption by 36.43%.

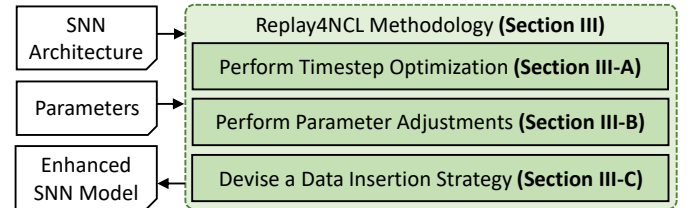


Fig. 3. An overview of our novel contributions; highlighted in green.

II. RELATED WORK

A. Spiking Neural Networks (SNNs)

SNNs are the emerging neural network (NN) models, that utilize action potentials (i.e., spikes) to convey information and perform computation. They offer ultra-low-power/energy consumption due to their sparse spike-based operations, while achieving high performance (e.g., accuracy) [31]–[33]. SNN computations typically rely on a specific neuron model. The widely-used model is the Leaky Integrate-and-Fire (LIF) neuron [34], whose internal dynamics can be expressed as follows.

$$\tau \frac{dV_{mem}(t)}{dt} = -(V_{mem}(t) - V_{rst}) + Z(t) \quad (1)$$

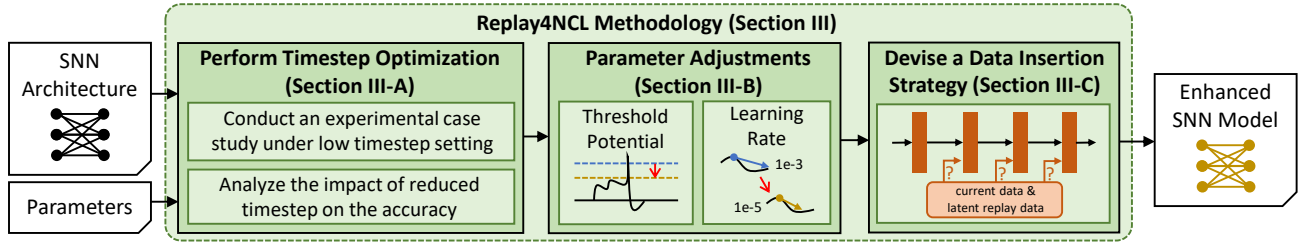


Fig. 4. Overview of our Replay4NCL methodology, showing its key steps such as timestep optimization, parameter adjustments, and data insertion strategy; the novel contributions are highlighted in green.

$$\text{if } V_{mem} \geq V_{thr} \text{ then } V_{mem} \leftarrow V_{rst} \quad (2)$$

V_{mem} is the neuron membrane potential at timestep- t , V_{rst} is the neuron reset potential, and τ is to the time constant of V_{mem} decay. Meanwhile, Z is the input, and V_{thr} is the neuron threshold potential. SNNs have advantages for their ultra-low power/energy consumption and temporal information processing by leveraging event-based data, and hence the SNN deployments on tightly-constrained embedded AI systems is highly desired.

B. Training SNNs with Surrogate Gradient (SG) Learning

The state-of-the-art SNN training under supervised settings typically involve the SG-based technique [35] [36] due to its efficacy in minimizing the loss function, e.g., using the Back-Propagation Through Time (BPTT) [37]. Therefore, in this work, we also train SNNs using the SG-based BPTT by using a fast-sigmoid surrogate gradient to approximate the step function, following the studies in [25]; see Fig. 5.

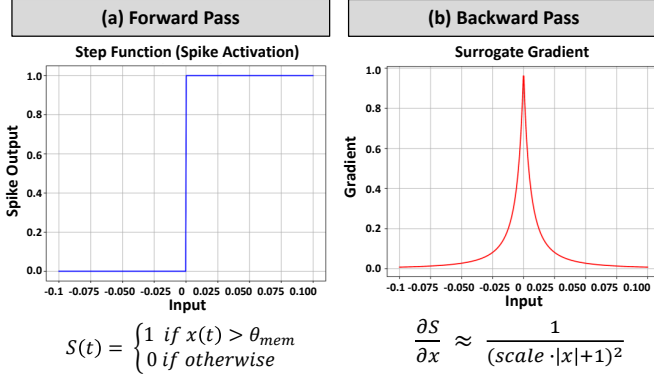


Fig. 5. Overview of (a) the forward pass, and (b) the backward pass with SG learning technique under supervised settings.

C. Memory Replay-based Techniques for NCL

The idea of these techniques is the generation of synthetic data samples of previously learned tasks, which will be re-introduced during the NCL training phase for updating the knowledge of the system [2] [12]. This approach presents new and old tasks to the system, thereby enforcing the system to re-learn old tasks (old knowledge) through the *latent replay (LR) data* to avoid CF issues, while also learning new tasks (new knowledge). To enable NCL with memory replay, we consider employing the SNN architecture from the SpikingLR work [25], which is illustrated in Fig. 6. The network comprises of *frozen* and *learning layers*, based on where the LR data is inserted into the network. For instance, if we have L

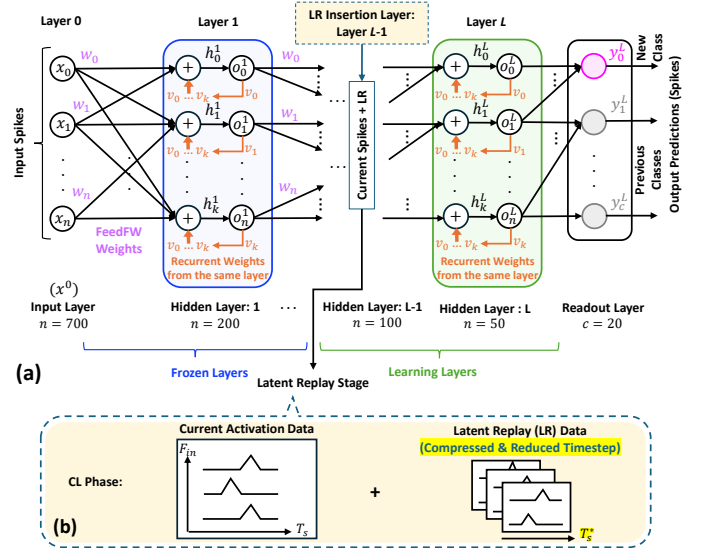


Fig. 6. Overview of (a) SNN architecture considered in this work, and (b) configuration of current activation data and latent replay (LR) data.

layers in the network and the LR data is inserted in layer $L-1$, then layer 1 until layer $L-2$ are the frozen layers, while layer $L-1$ and L are the learning layers. During the training on new data, only the learning layers are updated, while the frozen layers remain unchanged. These frozen layers simply forward the input spikes by processing them based on the pre-training phase. In this work, we consider a 4-layer SNN (i.e., $L = 4$).

III. PROPOSED REPLAY4NCL METHODOLOGY

Our proposed methodology, illustrated in Fig. 4, employs several key steps, as outlined below to enable efficient memory replay-based NCL.

A. Perform Timestep Optimization

This step aims at optimizing the timestep setting for SNN processing, thereby substantially reducing the processing latency. To effectively perform such an optimization, we conduct an experimental case study that considers different timestep settings, and then analyze their impact on accuracy and latency. This way, we can identify the appropriate timestep settings that can provide high accuracy without any enhancements. Furthermore, to minimize the memory size of LR data (i.e., latent memory), we also employ a data compression-decompression mechanism based on the reduced timestep (see Fig. 7), thereby optimizing both the timestep and memory requirements while preserving information for NCL training phase.

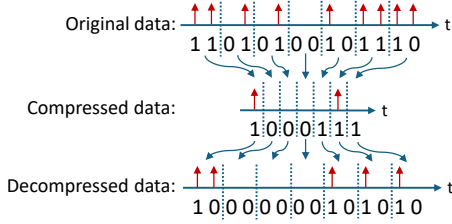


Fig. 7. A compression and decompression mechanism considered in this work; adopted from studies in [25].

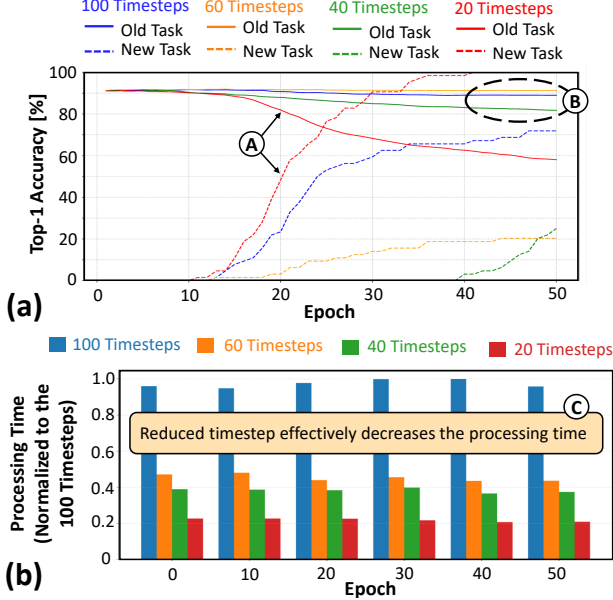


Fig. 8. Profiles of (a) accuracy and (b) processing latency across training epochs for different timestep settings (i.e., 100, 60, 40, and 20). Here, the timestep=100 represents the setting used by the state-of-the-art [25].

The experimental results are provided in Fig. 8, from which we draw several key observations below.

- Observation ①: An aggressive timestep reduction from 100 to 20 timesteps significantly reduces the accuracy of old tasks (old knowledge).
- Observation ②: To obtain acceptable accuracy for the old tasks (old knowledge) comparable to the state-of-the-art, we should employ at least 40 timesteps.
- Observation ③: Reduced timestep effectively decreases the processing time (latency), but they should be leveraged carefully to avoid significant accuracy degradation.

These observation points are leveraged for devising effective parameter adjustments in Section III-B.

B. Parameter Adjustments

This step aims at compensating the information loss due to the reduced timestep. Here, we observe that the generated spikes are less as compared to the original pre-trained model. The reason is that, neurons still employ the same threshold potential V_{thr} like the pre-trained model, but they encounter fewer number of spikes, which may not be enough for increasing the membrane potential V_{mem} to reach the threshold V_{thr} . Therefore, to make the network model adapt better to fewer spikes, we adjust the values of threshold potential V_{thr} and learning rate η with the following policy.

- **Threshold Potential (V_{thr}):** We adjust V_{thr} by reducing its value, since it will decrease the number of incoming spikes required to make V_{mem} reach V_{thr} , and hence preserving the spiking activity closer to the original pre-trained model.
- **Learning Rate (η):** We adjust η also by reducing its value, as it will decrease the learning speed of the network. This will ensure more careful weight updates during the training phase, considering that the network needs to leverage fewer spikes compared to the original pre-trained model.

C. Devise a Latent Replay Data Insertion Strategy

This step aims at devising a strategy to effectively benefit from the LR data insertion into the network. To achieve this, we perform a design exploration to see the impact of LR data insertion into different layers, and select the one that offers the highest accuracy. Additionally, this exploration also includes timestep optimization and parameter adjustments from Sections III-A and III-B. To incorporate all these techniques into an integrated mechanism, we propose a training strategy, whose key steps are presented in Alg. 1 and discussed below.

- First, we perform the pre-training phase, where the SNN model is trained and developed to learn all tasks from pre-training dataset; see Alg. 1: line 1-5.
 - Second, we prepare the network for NCL training phase. To do this, we perform the following.
 - We generate the LR data activations (A_{LR}), and then split the SNN model based on the selected layer for LR data insertion (i.e., so-called *LR layer*), following the approach from [25]; see Alg. 1: line 6-20.
 - We propose to dynamically change the threshold potential V_{thr} based on the timestep, adjustment interval, and spike timing information. If the spikes occur during the defined interval, V_{thr} is increased; otherwise V_{thr} is decreased using a sigmoidal decay; see Alg. 1: line 10-17. This way, the generated A_{LR} adapt better to the reduced timestep.
 - Third, we perform the NCL training phase by learning new tasks, while considering the following actions.
 - We dynamically adjust V_{thr} based on the timestep, adjustment interval, and spike timing information. This adaptive V_{thr} is updated based on the spike timing. If the spikes occur during the defined interval, V_{thr} is increased; otherwise V_{thr} is decreased; see Alg. 1: line 25-30.
 - We also reduce the learning rate η to make the network learn from fewer spikes more carefully; see Alg. 1: 21-33.
- As a result, the trained network is considered as the final output; see Alg. 1: 34-35.

IV. EVALUATION METHODOLOGY

Fig. 9 illustrates the experimental setup for evaluating our Replay4NCL methodology. Here, we employ a Python-based implementation that runs on the Nvidia RTX 4090 Ti GPU machines with the SHD dataset [30] as the workload. For the network architecture, we use the SNN architecture shown in Fig. 6 with 4 layers. We also consider the class-incremental scenario, i.e., the SNN is trained with 19 tasks (classes) during the pre-training phase, and then it is trained with 20th task

Algorithm 1 LR Data Training Strategy

Require: [1] Training set: pre-training (TS_{pre}), continual learning (TS_{cl}), subset of the pre-training (TS_{replay});
[2] Network parameters: network (net), number of layers (L), threshold potential (V_{thr}), learning rate (η), insertion layer for LR data (L_{ins}), reduced timesteps (T_{step}), adjust_interval=5;
[3] Epoch: pre-training (E_{pre}), continual learning (E_{cl});
[4] Activations: current (A_{new}), LR (A_{LR});
Ensure: Enhanced SNN model (SNN);

BEGIN
Pre-training phase:
1: Initialize SNN weights w and v randomly;
2: $\eta_{pre} = 1e-3$;
3: **for** ($ep=0$; $ep < E_{pre}$; $ep++$) **do**
4: $w, v \leftarrow \text{train}_{pre}(net, (w, v), TS_{pre}, \eta_{pre}, V_{th})$;
5: **end for**
Network preparation for NCL training phase:
6: $\eta_{cl} = \eta_{pre}/100$;
7: $net_f, net_l = \text{split}(net, L_{ins})$;
8: **for** ($ep=0$; $ep < E_{pre}$; $ep++$) **do**
9: **for** ($t=0$; $t < T_{step}$; $t++$) **do**
10: **if** ($t \% \text{adjust_interval} == 0$) **then**
11: **if** any spikes occur **then**
12: $\text{avg_spike_time} = \text{mean}(\text{spike_timing})$;
13: $V_{thr} = 1 + 0.01(T_{step} - \text{avg_spike_time})$;
14: **end if**
15: **else**
16: $V_{thr} = 1 / (1 + \exp(-0.001 \cdot t))$;
17: **end if**
18: $A_{LR} = \text{infer}(net_f, TS_{replay} \subseteq TS_{pre})$;
19: **end for**
20: **end for**
NCL training phase:
21: $\eta_{cl} = \eta_{pre}/100$;
22: **for** ($ep=0$; $ep < E_{cl}$; $ep++$) **do**
23: $A_{new} = \text{inference}(net_f, TS_{cl})$;
24: **for** ($t=0$; $t < T_{step}$; $t++$) **do**
25: **if** any spikes occur **then**
26: $\text{avg_spike_time} = \text{mean}(\text{spike_timing})$;
27: $V_{thr} = 1 + 0.01(T_{step} - \text{avg_spike_time})$;
28: **else**
29: $V_{thr} = 1 / (1 + \exp(-0.001 \cdot t))$;
30: **end if**
31: $w_l, v_l \leftarrow \text{train}(net_l, (w_l, v_l), A_{new} \cup A_{LR}, \eta_{cl}, V_{thr})$;
32: **end for**
33: **end for**
34: $SNN \leftarrow net$;
35: **return** SNN ;
END

(class) during the CL training phase. For the comparison partner, we consider the SpikingLR with its compressed LR data technique [25] as the state-of-the-art. The evaluation generates several results, including the accuracy for new and old tasks, latency, latent memory size, and energy consumption.

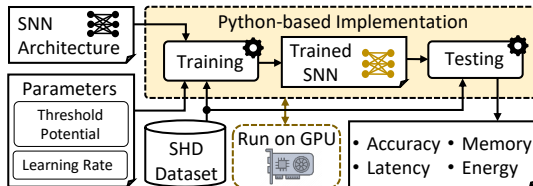


Fig. 9. Experimental setup considered in this work.

V. EXPERIMENTAL RESULTS AND DISCUSSION

A. Maintaining High Accuracy for Old and New Tasks

Experimental results for accuracy across different LR insertion layers are shown in Fig. 10(a). In general, we observe

that our Replay4NCL achieves comparable accuracy to the SpikingLR across different LR insertion layers for learning a new task. Moreover, for the LR insertion layers 0-2, our Replay4NCL achieves 100% accuracy when learning a new task within 50 epochs, which is better than the SpikingLR. For the LR insertion layer 3, the Replay4NCL achieves comparable accuracy to the SpikingLR when learning a new task (i.e., 67.19% for Replay4NCL and 71.88% for the SpikingLR); see ①. Meanwhile, when learning old tasks, our Replay4NCL achieves comparable high accuracy to the SpikingLR in the LR insertion layer 3 (i.e., 88.24% for Replay4NCL and 90.19% for the state-of-the-art); see ①. These results indicate that the Replay4NCL maintains high accuracy comparable to the SpikingLR on both learning the old and new tasks, despite employing low timestep settings. The reason is the Replay4NCL employs parameter adjustments to compensate the information loss due to fewer spikes, and leverages a training strategy that determines the effective threshold potential V_{thr} and learning rate η values.

Meanwhile, experimental results for accuracy across different training epochs considering the LR insertion layer 3, are provided in Fig. 11(a). We observe that Replay4NCL achieves higher Top-1 accuracy than the SpikingLR (see ④), since its parameter adjustments and training strategy effectively improve the learning process on the later layers.

B. Improvements on Processing Latency

Experimental results for processing time (latency) across different LR insertion layers are provided in Fig. 10(b). In general, our Replay4NCL significantly reduces the processing time as compared to the SpikingLR, i.e., by up to 2.34x speed-up; see ②. Similar trends are also observed for processing latency in the LR insertion layer 3 case, as shown by ⑤ in Fig. 11(b). These improvements are obtained mainly due to the reduction of timestep settings.

C. Latent Memory Savings

Experimental results for the latent memory size considering different LR insertion layers are provided in Fig. 12. In general, we observe that later layers tend to require smaller latent memory sizes due to smaller layer dimension (i.e., fewer number of neurons). Furthermore, these results also show that our Replay4NCL significantly saves the latent memory size as compared to the SpikingLR by 20%-21.88%, due to the timestep reduction which decreases the size of spike-based LR data activations to be stored in the memory. Such memory savings are beneficial for ensuring the applicability of our Replay4NCL for tightly-constrained embedded AI systems.

D. Reduction on Energy Consumption

Experimental results for energy consumption across different LR insertion layers are provided in Fig. 10(c). In general, we observe that our Replay4NCL significantly decreases the energy consumption from the SpikingLR across different LR insertion layers. Specifically, the Replay4NCL offers up to 56.7% energy savings; see ③. Similar trends are also observed for energy consumption in the LR insertion layer 3, as the Replay4NCL offers 36.4% energy saving; see ⑥ in Fig. 11(c).

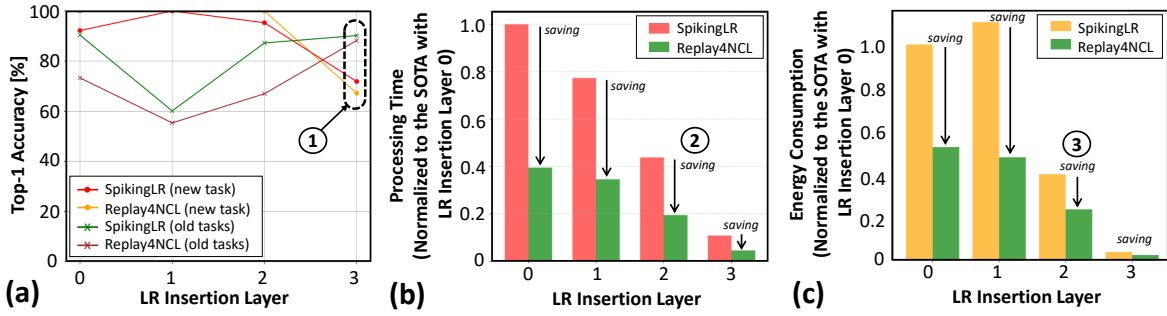


Fig. 10. Results for (a) Top-1 accuracy considering both old and new tasks, (b) processing time, and (c) energy consumption of the SpikingLR and our Replay4NCL across different LR insertion layers.

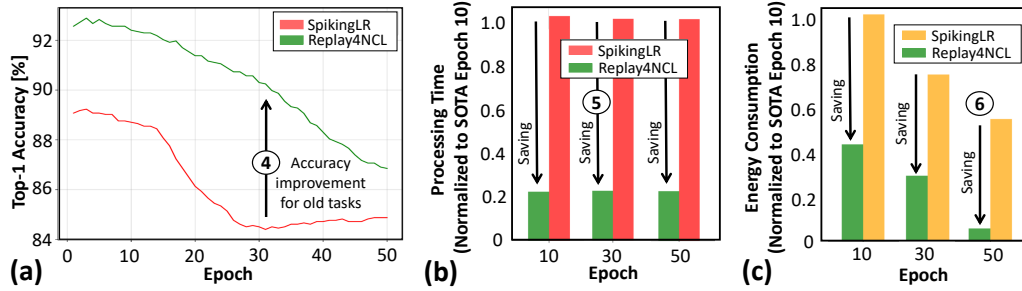


Fig. 11. Results for (a) Top-1 accuracy of old tasks, (b) processing time, and (c) energy consumption of the SpikingLR and our Replay4NCL across different training epochs considering the LR insertion layer 3.

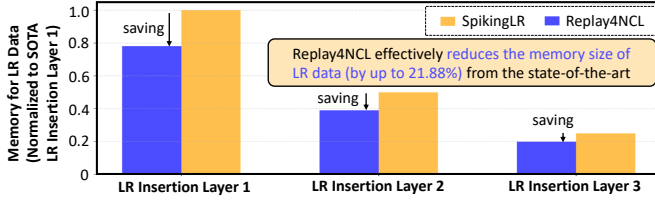


Fig. 12. Comparison of the latent memory sizes between the SpikingLR and our Replay4NCL across different LR insertion layers.

The reason is that, our Replay4NCL employs short timestep settings which significantly reduce the processing latency, thereby leading to low energy consumption.

E. Further Discussion

We further investigate the accuracy of the SpikingLR and our Replay4NCL considering a long training period (i.e., 150 epochs), and the experimental results are shown in Fig. 13. These results show that our Replay4NCL can achieve higher accuracy than the SpikingLR, as shown by ⑦. This indicates that the proposed LR training strategy in Replay4NCL provides an effective integrated mechanism to maximize the benefits from the proposed optimization through timestep reduction, and the NCL training enhancements through parameter adjustments. Moreover, with the employment of a lower learning rate η than the SpikingLR, our Replay4NCL offers more careful weight updates, which in turns leading to better learning convergence when processing fewer spikes as indicated by a smoother learning curve.

VI. CONCLUSION

In this paper, we propose the Replay4NCL methodology to enable efficient memory replay for NCL in embedded AI

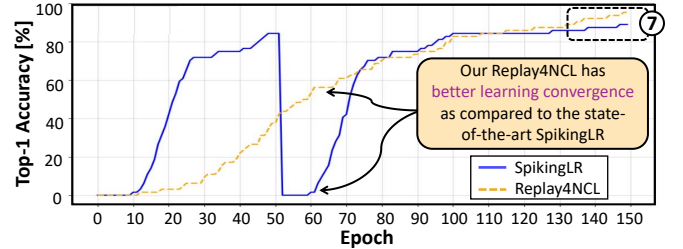


Fig. 13. Comparison of accuracy profiles between the SpikingLR and our Replay4NCL when learning a new task, while considering a long training period, i.e., 150 training epochs.

systems. Our Replay4NCL compresses LR data (old knowledge) and replays them during the CL training phase using small timestep settings, hence reducing latency and energy consumption. To compensate the information loss due to fewer spikes, we adapt the neuron threshold potential and learning rate settings. Experimental results on the class-incremental scenario with the SHD dataset demonstrate that Replay4NCL effectively supports the new task learning, while avoiding CF problem. Specifically, Replay4NCL maintains old knowledge with Top-1 accuracy of 90.43% compared to the state-of-the-art with 86.22%, while improving the latency by 4.88x, saving the latent memory by 20%, and saving the energy consumption by 36.43%. These results demonstrate the potential of our Replay4NCL methodology for enabling NCL for tightly-constrained embedded AI systems.

ACKNOWLEDGMENT

This work was partially supported by United Arab Emirates University (UPAR) with Fund code 12N169. This work was also partially supported by the NYUAD Center for Artificial Intelligence and Robotics (CAIR), funded by Tamkeen under the NYUAD Research Institute Award CG010.

REFERENCES

- [1] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *International conference on machine learning*. PMLR, 2017, pp. 3987–3995.
- [2] L. Wang, X. Zhang, H. Su, and J. Zhu, "A comprehensive survey of continual learning: Theory, method and application," *IEEE Transactions on Pattern Analysis and Machine Intelligence (PAMI)*, 2024.
- [3] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars, "A continual learning survey: Defying forgetting in classification tasks," *IEEE transactions on pattern analysis and machine intelligence*, vol. 44, no. 7, pp. 3366–3385, 2021.
- [4] R. V. W. Putra, M. A. Hanif, and M. Shafique, "Rescuesnn: enabling reliable executions on spiking neural network accelerators under permanent faults," *Frontiers in Neuroscience*, vol. 17, p. 1159440, 2023.
- [5] R. M. French, "Catastrophic forgetting in connectionist networks," *Trends in cognitive sciences*, vol. 3, no. 4, pp. 128–135, 1999.
- [6] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," vol. 24, pp. 109–165, 1989.
- [7] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska et al., "Overcoming catastrophic forgetting in neural networks," *Proceedings of the national academy of sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [8] G. M. van de Ven, N. Soures, and D. Kudithipudi, "Continual learning and catastrophic forgetting," *arXiv preprint arXiv:2403.05175*, 2024.
- [9] A. Iyer, K. Grewal, A. Velu, L. O. Souza, J. Forest, and S. Ahmad, "Avoiding catastrophe: Active dendrites enable multi-task learning in dynamic environments," *Frontiers in neurorobotics*, vol. 16, p. 846219, 2022.
- [10] S.-W. Lee, J.-H. Kim, J. Jun, J.-W. Ha, and B.-T. Zhang, "Overcoming catastrophic forgetting by incremental moment matching," in *Advances in neural information processing systems (NIPS)*, 2017, pp. 4652–4662.
- [11] R. V. W. Putra and M. Shafique, "Mantis: Enabling energy-efficient autonomous mobile agents with spiking neural networks," in *2023 9th International Conference on Automation, Robotics and Applications (ICARA)*, 2023, pp. 197–201.
- [12] M. F. Minhas, R. V. W. Putra, F. Awwad, O. Hasan, and M. Shafique, "Continual learning with neuromorphic computing: Theories, methods, and applications," *arXiv preprint arXiv:2410.09218*, 2024.
- [13] P. Panda, J. M. Allred, S. Ramanathan, and K. Roy, "Asp: Learning to forget with adaptive synaptic plasticity in spiking neural networks," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems (JETCAS)*, vol. 8, no. 1, 2018.
- [14] J. M. Allred and K. Roy, "Controlled forgetting: Targeted stimulation and dopaminergic plasticity modulation for unsupervised lifelong learning in spiking neural networks," *Frontiers in Neuroscience*, vol. 14, 2020.
- [15] R. V. W. Putra and M. Shafique, "Spikedyn: A framework for energy-efficient spiking neural networks with continual and unsupervised learning capabilities in dynamic environments," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 1057–1062.
- [16] —, "Ipspikecon: Enabling low-precision spiking neural network processing for efficient unsupervised continual learning on autonomous agents," in *2022 International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8.
- [17] M. Davies, N. Srinivasa, T. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain, Y. Liao, C. Lin, A. Lines, R. Liu, D. Mathaikutty, S. McCoy, A. Paul, J. Tse, G. Venkataramanan, Y. Weng, A. Wild, Y. Yang, and H. Wang, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, Jan 2018.
- [18] A. Ororbia, "Spiking neural predictive coding for continually learning from data streams," *Neurocomputing*, vol. 544, p. 126292, 2023.
- [19] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [20] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.
- [21] L. Pes, R. Luiken, F. Corradi, and C. Frenkel, "Active dendrites enable efficient continual learning in time-to-first-spike neural networks," *arXiv preprint arXiv:2404.19419*, 2024.
- [22] B. Han, F. Zhao, Y. Zeng, W. Pan, and G. Shen, "Enhancing efficient continual learning with dynamic structure development of spiking neural networks," *arXiv preprint arXiv:2308.04749*, 2023.
- [23] B. Han, F. Zhao, W. Pan, Z. Zhao, X. Li, Q. Kong, and Y. Zeng, "Adaptive reorganization of neural pathways for continual learning with hybrid spiking neural networks," *arXiv preprint arXiv:2309.09550*, 2023.
- [24] M. Proietti, A. Ragno, and R. Capobianco, "Memory replay for continual learning with spiking neural networks," in *2023 IEEE 33rd International Workshop on Machine Learning for Signal Processing (MLSP)*. IEEE, 2023, pp. 1–6.
- [25] A. Dequino, A. Carpegna, D. Nadalini, A. Savino, L. Benini, S. Di Carlo, and F. Conti, "Compressed latent replays for lightweight continual learning on spiking neural networks," in *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2024, pp. 240–245.
- [26] D. Antonov, K. Sviatov, and S. Sukhov, "Continuous learning of spiking networks trained with local rules," *Neural Networks*, vol. 155, pp. 512–522, 2022.
- [27] N. Skatchkovsky, H. Jang, and O. Simeone, "Bayesian continual learning via spiking neural networks," *Frontiers in Computational Neuroscience*, vol. 16, p. 1037976, 2022.
- [28] M. Xiao, Q. Meng, Z. Zhang, D. He, and Z. Lin, "Hebbian learning based orthogonal projection for continual learning of spiking neural networks," *arXiv preprint arXiv:2402.11984*, 2024.
- [29] R. V. W. Putra and M. Shafique, "Topspark: A timestep optimization methodology for energy-efficient spiking neural networks on autonomous mobile agents," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 3561–3567.
- [30] B. Cramer, Y. Stradmann, J. Schemmel, and F. Zenke, "The heidelberg spiking data sets for the systematic evaluation of spiking neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 7, pp. 2744–2757, 2020.
- [31] A. Tavanaei, M. Ghodrati, S. R. Kheradpisheh, T. Masquelier, and A. Maida, "Deep learning in spiking neural networks," *Neural Networks*, vol. 111, pp. 47–63, 2019.
- [32] J. D. Nunes, M. Carvalho, D. Carneiro, and J. S. Cardoso, "Spiking neural networks: A survey," *IEEE Access*, vol. 10, pp. 60 738–60 764, 2022.
- [33] R. V. W. Putra and M. Shafique, "FSpiNN: An optimization framework for memory-efficient and energy-efficient spiking neural networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 39, no. 11, pp. 3601–3613, 2020.
- [34] E. M. Izhikevich, "Which model to use for cortical spiking neurons?" *IEEE Trans. on Neural Networks (TNN)*, vol. 15, no. 5, pp. 1063–1070, Sep. 2004.
- [35] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Processing Magazine (MSP)*, vol. 36, no. 6, pp. 51–63, 2019.
- [36] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, "Spatio-temporal backpropagation for training high-performance spiking neural networks," *Frontiers in Neuroscience*, vol. 12, 2018.
- [37] P. J. Werbos, "Backpropagation through time: what it does and how to do it," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.